

5.5.25

Chiruvella Harshith Sharan - AI25BTECH11008

October 3, 2025

Question

Solve the system using vector algebra:

$$x + y + z = 6$$

$$2x + 3y + z = 10$$

$$x - y + 2z = 5$$

Step 1: Matrix Form

System can be written as

$$A\vec{x} = \vec{b} \quad (1)$$

where

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 3 & 1 \\ 1 & -1 & 2 \end{bmatrix}, \quad \vec{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \quad \vec{b} = \begin{bmatrix} 6 \\ 10 \\ 5 \end{bmatrix}.$$

Step 2: Formula for Inverse

The solution is

$$\vec{x} = A^{-1}\vec{b} \quad (2)$$

where

$$A^{-1} = \frac{1}{\det(A)} \text{adj}(A).$$

Step 3: Computing Inverse

$$\det(A) = 7 \neq 0$$

So inverse exists. After computation,

$$A^{-1} = \begin{bmatrix} 5/7 & 1/7 & -3/7 \\ 1 & -1/7 & -1/7 \\ -2/7 & 2/7 & 3/7 \end{bmatrix} \quad (3)$$

Step 4: Solution

$$\vec{x} = A^{-1}\vec{b} = \begin{bmatrix} 5/7 & 1/7 & -3/7 \\ 1 & -1/7 & -1/7 \\ -2/7 & 2/7 & 3/7 \end{bmatrix} \begin{bmatrix} 6 \\ 10 \\ 5 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \quad (4)$$

Final Answer

$$\vec{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

Python Code

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np

# Define column vectors of A
a1 = np.array([1,2,1])
a2 = np.array([2,0,2])
a3 = np.array([-3,-3,0])
b = np.array([1,2,3])

# Scaled components from solution  $x = [2, 1/2, 2/3]$ 
v1 = 2*a1
v2 = 0.5*a2
v3 = (2/3)*a3

# Step-by-step sum points
origin = np.array([0,0,0])
p1 = v1
p2 = v1+v2
```


Python Code

```
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')

# Axes limits
ax.set_xlim(-4,4)
ax.set_ylim(-4,4)
ax.set_zlim(-1,4)

# Plot column vectors
ax.quiver(0,0,0, *a1, color='blue', label='a1')
ax.quiver(0,0,0, *a2, color='green', label='a2')
ax.quiver(0,0,0, *a3, color='red', label='a3')
ax.quiver(0,0,0, *b, color='purple', label='b')

# Plot scaled solution vectors
ax.quiver(0,0,0, *v1, color='blue', alpha=0.5)
ax.quiver(0,0,0, *v2, color='green', alpha=0.5)
```

Python Code

Labels

```
ax.text(*a1, "a1", color='blue')
ax.text(*a2, "a2", color='green')
ax.text(*a3, "a3", color='red')
ax.text(*b, "b", color='purple')
ax.text(*v1, "2a1", color='blue')
ax.text(*v2, "1/2 a2", color='green')
ax.text(*v3, "2/3 a3", color='red')
```

Axes labels and title

```
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("z")
ax.legend()
ax.set_title("3D visualization of  $2a_1 + (1/2)a_2 + (2/3)a_3 = b$ ")
```

```
plt.show()
```

C Code

```
#include <stdio.h>

#define N 3    // 3x3 matrix

// Function to calculate determinant of a 3x3 matrix
float determinant(float A[3][3]) {
    float det;
    det = A[0][0]*(A[1][1]*A[2][2] - A[1][2]*A[2][1])
        - A[0][1]*(A[1][0]*A[2][2] - A[1][2]*A[2][0])
        + A[0][2]*(A[1][0]*A[2][1] - A[1][1]*A[2][0]);
    return det;
}

// Function to find adjoint of 3x3 matrix
void adjoint(float A[3][3], float adj[3][3]) {
    adj[0][0] = (A[1][1]*A[2][2] - A[1][2]*A[2][1]);
    adj[0][1] = - (A[0][1]*A[2][2] - A[0][2]*A[2][1]);
    adj[0][2] = (A[0][1]*A[1][2] - A[0][2]*A[1][1]);
```

C Code

```
adj[1][0] = - (A[1][0]*A[2][2] - A[1][2]*A[2][0]);
adj[1][1] = (A[0][0]*A[2][2] - A[0][2]*A[2][0]);
adj[1][2] = - (A[0][0]*A[1][2] - A[0][2]*A[1][0]);

adj[2][0] = (A[1][0]*A[2][1] - A[1][1]*A[2][0]);
adj[2][1] = - (A[0][0]*A[2][1] - A[0][1]*A[2][0]);
adj[2][2] = (A[0][0]*A[1][1] - A[0][1]*A[1][0]);

// Transpose of cofactor matrix = adj(A)
float temp;
temp = adj[0][1]; adj[0][1] = adj[1][0]; adj[1][0]
    = temp;
temp = adj[0][2]; adj[0][2] = adj[2][0]; adj[2][0]
    = temp;
temp = adj[1][2]; adj[1][2] = adj[2][1]; adj[2][1]
    = temp;
```

```
}
```

C Code

```
// Function to find inverse of 3x3 matrix
int inverse(float A[3][3], float inv[3][3]) {
    float det = determinant(A);
    if (det == 0) {
        printf("Matrix is singular, no inverse exists
               .\n");
        return 0;
    }

    float adj[3][3];
    adjoint(A, adj);

    for (int i=0; i<3; i++)
        for (int j=0; j<3; j++)
            inv[i][j] = adj[i][j] / det;

    return 1;
}
```

C Code

```
// Multiply 3x3 matrix with 3x1 vector
void multiply(float A[3][3], float b[3], float x[3]) {
    for (int i=0; i<3; i++) {
        x[i] = 0;
        for (int j=0; j<3; j++)
            x[i] += A[i][j] * b[j];
    }
}

int main() {
    // Define A and b
    float A[3][3] = {
        {1, 2, -3},
        {2, 0, -3},
        {1, 2, 0}
    };
    float b[3] = {1, 2, 3};
```

```
float invA[3][3], x[3];

// Find inverse
if (inverse(A, invA)) {
    // Solve  $x = A^{-1} b$ 
    multiply(invA, b, x);

    printf("Solution:\n");
    printf("x = %.3f\n", x[0]);
    printf("y = %.3f\n", x[1]);
    printf("z = %.3f\n", x[2]);
}

return 0;
}
```

beamer/figs/fig1.png